

RELATÓRIO TÉCNICO - TRABALHO 4: Comunicação Indireta com Filas de Mensagens

Equipe: Jessica Clezia de Oliveira Campos, Victor Kauã Guilherme dos Santos

1. Justificativa da Escolha: Filas de Mensagens (Opção C)

A abordagem escolhida para este trabalho foi a Opção C, Filas de Mensagens (Message Queues). A decisão foi motivada pelas características específicas do sistema de Pixel Art colaborativo e pelos problemas de acoplamento identificados na arquitetura síncrona do Trabalho 3.

No Trabalho 3, toda operação no mural era processada de forma síncrona: o cliente envia a requisição, o servidor aplica imediatamente a mudança e retornava a resposta. Esse modelo funciona, mas cria um acoplamento temporal forte: se o componente de processamento estivesse sobrecarregado ou indisponível, a requisição falharia. No contexto do Pixel Art, isso se torna particularmente visível em operações de área com o Pincel, que podem gerar dezenas de mudanças em sequência.

A fila de mensagens resolve esse problema de forma natural para este domínio. As operações de edição do mural são inerentemente sequenciais e tolerantes a um pequeno atraso de processamento: o usuário envia o comando de pintar e não precisa aguardar que cada pixel seja persistido antes de enviar o próximo comando. Com a fila, todas as operações são enfileiradas instantaneamente pelo servidor e o consumidor (WorkerFile) as processa no seu próprio ritmo, sem bloquear o cliente.

Em comparação com as outras opções disponíveis, a fila é a mais adequada para este cenário. O Multicast (Opção A) pressupõe múltiplos receptores que precisam sincronizar o estado, o que não se aplica a um mural centralizado. O Pub-Sub (Opção B) adicionaria complexidade de tópicos e subscrições desnecessária para um único consumidor. O Tuple Space (Opção DE) introduziria semântica de leitura associativa incompatível com o modelo sequencial e ordenado de edição de pixels.

2. Implementação da Comunicação Indireta

A fila de mensagens foi implementada de forma caseira, sem bibliotecas externas como RabbitMQ ou Kafka, utilizando um arquivo de texto simples como mecanismo de persistência. Essa escolha demonstra o entendimento dos conceitos fundamentais sem depender de infraestrutura externa.

2.1 Fila Persistente e Eventos Fila

A classe Fila Persistente gerencia o arquivo dados/fila eventos.txt, onde cada linha representa um evento serializado em JSON. A operação publicar(Evento Fila) faz append de uma nova linha ao arquivo, com custo $O(1)$. A operação consumir() lê a primeira linha (garantindo ordenação FIFO), reescreve o arquivo sem ela e retorna o evento desserializado. A segurança concorrente entre os dois processos (Servidor API e Worker Fila rodando simultaneamente) é garantida pelo uso de `java.nio.channels.FileLock` sobre um arquivo de lock dedicado (dados/fila eventos.lock). Cada operação adquire o lock exclusivo, executa automaticamente e o libera, prevenindo corrupção de dados mesmo sob concorrência.

A classe Evento Fila encapsula todos os dados necessários para qualquer tipo de operação com uma representação unificada de campos, alguns opcionais conforme o tipo. Os cinco tipos de evento implementados são: CRIAR MURAL (campos largura, altura, dono), PINTAR PIXEL (campos x, y, cor), APAGAR PIXEL (campos x, y), APLICAR PINCEL (campos x1, y1, x2, y2, cor) e APLICAR BORRACHA (campos x, y). Cada evento também carrega o timestamp de criação (criado) para rastreabilidade.

2.2 Worker File e Mural Store

O Worker File é um processo Java independente, com seu próprio método main, que implementa o papel de consumidor na arquitetura de fila. Seu loop principal tenta consumir um evento da fila a cada ciclo: se houver evento pendente, carrega o estado atual do Mural via Mural Store, aplica a operação correspondente ao tipo do evento por meio de um switch, salva o Mural atualizado de volta ao Mural Store e aguarda 500

milissegundos antes do próximo ciclo. Se a fila estiver vazia, o Worker aguarda 1500 milissegundos antes de tentar novamente, reduzindo o consumo de CPU em períodos ociosos.

O Mural Store foi introduzido no Trabalho 4 para persistir o estado do Mural entre sessões do Worker. Ele utiliza serialização binária Java (ObjectInputStream e ObjectOutputStream) para gravar e recuperar o objeto Mural do arquivo dados/mural.estado.ser. Isso garante que, mesmo se o Worker for reiniciado ou falhar, o estado do mural seja recuperado automaticamente ao retornar, sem nenhuma perda dos pixels já processados.

3. Análise: Desacoplamento, Overhead e Limitações

3.1 Desacoplamento Espacial e Temporal

O desacoplamento espacial foi completamente obtido. O Servidor API publica eventos na fila (arquivo local compartilhado) sem qualquer referência ao IP, porta ou existência do Worker File. Os dois componentes se comunicam exclusivamente por meio do intermediário, sem nenhum canal ponto a ponto entre eles.

O desacoplamento temporal é a propriedade mais relevante para este trabalho e pode ser demonstrada em tempo de execução da seguinte forma: encerrar o Worker File, realizar diversas operações pelos clientes (os eventos são publicados normalmente na fila), e então reiniciar o Worker. Ao retornar, o Worker processa todos os eventos pendentes em sequência, atualizando o mural com todas as mudanças que ocorreram enquanto estava offline. O servidor API responde normalmente durante todo esse período, sem nenhuma indicação de falha ao cliente, exceto pelo campo mensagens Pendentes retornado nas consultas de visualização.

3.2 Overhead Introduzido e Limitações

A introdução do intermediário adiciona várias fontes de overhead ao sistema em relação ao Trabalho 3. A primeira é o custo de I/O por operação: cada publicação e cada consumo exigem acesso ao disco. Essa sobrecarga foi mitigada usando append para publicação (operação rápida) e reescrita completa apenas no momento do consumo. Para volumes muito altos de operações simultâneas, substituir a fila em arquivo por um broker dedicado como RabbitMQ eliminaria esse gargalo.

A segunda fonte de overhead é o FileLock entre processos: o bloqueio exclusivo aumenta a latência das operações de fila em cenários de alta concorrência entre produtor e consumidor. A mitigação adotada é manter o lock pelo menor tempo possível, limitando a seção crítica a apenas a operação atômica sobre o arquivo. O pool de dez threads do Servidor API também absorve múltiplas requisições simultâneas sem bloquear umas às outras.

A terceira e mais relevante fonte de overhead é a visibilidade assíncrona do estado: o cliente que pinta um pixel recebe imediatamente a confirmação de que o evento foi enfileirado, mas o mural retornado por uma consulta subsequente pode ainda não refletir essa mudança se o Worker ainda não a processa. Para mitigar essa confusão, todas as respostas de consulta do mural incluem o campo mensagens Pendentes, informando quantos eventos aguardam processamento. Assim o usuário sabe que o mural visualizado pode estar ligeiramente desatualizado.

4. Evolução em Relação ao Trabalho 3

Além da introdução da fila de mensagens, o Trabalho 4 trouxe um novo endpoint GET /fila/status, tratado pelo File Resource, que retorna o número de eventos pendentes na fila no formato {"ok":true,"eventos Pendentes":N}. Esse endpoint é útil para monitoramento e para demonstrar o desacoplamento temporal em tempo real. Ambos os clientes (Java e Python) foram atualizados com uma opção f8 no menu para consultar esse status. O cliente Python também passou a exibir o número de mensagens pendentes ao visualizar o mural, reforçando a consciência do usuário sobre o estado assíncrono do sistema.

5. Conclusão

O Trabalho 4 evoluiu a arquitetura do sistema Mural Pixel Art de uma comunicação síncrona direta para uma arquitetura assíncrona baseada em filas de mensagens. A escolha da Opção C foi adequada ao contexto do projeto: o sistema de edição de Pixel Art se beneficia do processamento assíncrono, especialmente para

operações em lote como o pincel de área, e a semântica FIFO da fila garante que as mudanças sejam aplicadas na ordem correta.

A implementação caseira da fila persistente demonstra os conceitos fundamentais sem depender de infraestrutura externa: intermediário de mensagens com persistência em arquivo, publicação atômica com append, consumo FIFO, lock exclusivo para segurança concorrente entre processos e persistência do estado do consumidor. Os dois requisitos centrais do trabalho, desacoplamento espacial e desacoplamento temporal, foram plenamente implementados e são demonstráveis em execução.